

Лабораторная работа №2

Егор Мальцев

Z33434

Цель

Познакомить студента с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомление с типами данных.

Задачи

1. [C++ EXPRESSION] Создать и скомпилировать программу на C++

Результат сборки (компиляции) сохранять в папку build. Папку build сделать игнорируемой для GIT. Программа должна получать на вход число – это количество итераций для выполнения расчета. В рамках итерации выполнять следующее вычисление: $x^2 - x^2 + x^4 - x^5 + x + x$. Вычисление выполнять в виде отдельной от main функции, которая будет вызвана циклически из main. Фиксировать время выполнения программы, затрачиваемое на расчет выражения n раз (n задается в консоли перед вычислением). Предусмотреть дополнительный цикл на повторную итерацию запуска программы вычислений. Если было введено не число, то завершить выполнение программы.

2. [PYTHON EXPRESSION] Создать и скомпилировать программу на Python 3

Результат сборки (компиляции) сохранять в папку build. Папку build сделать игнорируемой для GIT. Программа должна получать на вход число – это количество итераций для выполнения расчета. В рамках итерации выполнять следующее вычисление: $x^2 - x^2 + x^4 - x^5 + x + x$. Вычисление выполнять в виде отдельной от main функции, которая будет вызвана циклически из main. Фиксировать время выполнения программы, затрачиваемое на расчет выражения n раз (n задается в консоли перед вычислением). Предусмотреть дополнительный цикл на повторную итерацию запуска программы вычислений. Если было введено не число, то завершить выполнение программы.

3. [SAVE] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

Выполнение

C++

```
1  #include <iostream>
2  #include <string>
3  #include <chrono>
4
5  int Calculate(int x) {
6      return x * x - x * x + x * 4 - x * 5 + x + x;
7  }
8
9  int main() {
10     while (true) {
11         std::string str;
12         std::cin >> str;
13
14         for (char ch : str) {
15             if (ch < '0' || ch > '9') {
16                 return 0;
17             }
18         }
19
20         int n = std::stoi(str);
21         int x = 2;
22         auto start = std::chrono::high_resolution_clock::now();
23         for (int i = 0; i != n; ++i) {
24             x = Calculate(x);
25         }
26         auto end = std::chrono::high_resolution_clock::now();
27         auto diff = end - start;
28         std::cout << std::chrono::duration<double, std::chrono::seconds::period>(diff).count() << " s\n";
29     }
30 }
31
```

Python

```
1  import time
2
3  def Calculate(x):
4      return x * x - x * x + x * 4 - x * 5 + x + x
5
6  def isint(s):
7      try:
8          int(s)
9          return True
10     except ValueError:
11         return False
12
13  while True:
14      x = 2
15      n = input()
16      if not isint(n):
17          break
18      n = int(n)
19      start = time.time()
20      for _ in range(n):
21          x = Calculate(x)
22      finish = time.time()
23      t = finish - start
24      print(t)
```

Время выполнения (для n=1000000 итераций):

C++ t = 0.0024 s

Python t = 0.2137 s

Заключение

1. Убедился, что программа написанная на C++ работает гораздо быстрее.
2. Кратко ознакомился с библиотекой chrono